Boston Housing Price Prediction - Jupyter Notebook Code

1. Install Required Libraries

```
!pip install numpy pandas matplotlib seaborn scikit-learn tensorflow plotly
```

2. Import Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

3. Load Dataset

```
boston = fetch_openml(name="boston", version=1, as_frame=True)

data = boston.frame
data.head()
```

4. Rename Target Column

```
data.rename(columns={"MEDV": "PRICE"}, inplace=True)

data.head(10)
```

5. Shape of Dataset

```
print(data.shape)
```

6. Check Null Values

```
data.isnull().sum()
```

7. Dataset Information

```
data.info()
```

8. Statistical Summary

```
data.describe()
```

9. Distribution Plot

```
sns.histplot(data["PRICE"], kde=True)
plt.title("Distribution of House Prices")
```

```
plt.show()
```

10. Boxplot

```
sns.boxplot(x=data["PRICE"])
plt.title("Boxplot of House Prices")
plt.show()
```

11. Correlation Matrix

```
correlation = data.corr(numeric_only=True)
correlation["PRICE"]
```

12. Heatmap

```
plt.figure(figsize=(15, 12))
sns.heatmap(correlation, annot=True, square=True, cmap="coolwarm")
plt.title("Correlation Heatmap")
plt.show()
```

13. Scatter Plots

```
features = ["LSTAT", "RM", "PTRATIO"]

plt.figure(figsize=(20, 5))

for i, col in enumerate(features):
    plt.subplot(1, len(features), i + 1)
    plt.scatter(data[col], data["PRICE"])
    plt.title("Variation in House Prices")
    plt.xlabel(col)
    plt.ylabel("House Prices in $1000")

plt.show()
```

14. Split Features and Target

```
X = data.drop("PRICE", axis=1)
y = data["PRICE"]
```

15. Train Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

16. Feature Scaling

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

17. Linear Regression Model

```
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)

y_pred_lr = lr_model.predict(X_test)
mse_lr = mean_squared_error(y_test, y_pred_lr)
```

```
mae_lr = mean_absolute_error(y_test, y_pred_lr)
rmse_lr = np.sqrt(mse_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print("Linear Regression Results")
print("MSE:", mse_lr)
print("MAE:", mae_lr)
print("RMSE:", rmse_lr)
print("R2 Score:", r2_lr)
```

18. Deep Neural Network Model

```
model = Sequential()

model.add(Dense(128, activation="relu", input_dim=13))
model.add(Dense(64, activation="relu"))
model.add(Dense(32, activation="relu"))
model.add(Dense(16, activation="relu"))
model.add(Dense(1))

model.compile(
    optimizer="adam",
    loss="mean_squared_error",
    metrics=["mae"]
)

model.summary()
```

19. Train Model

```
history = model.fit(
    X_train,
    y_train,
    epochs=100,
    validation_split=0.05,
    verbose=1
)
```

20. Plot Training Loss

```
plt.figure(figsize=(8, 5))
plt.plot(history.history["loss"], label="Train Loss")
plt.plot(history.history["val_loss"], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Model Loss")
plt.legend()
plt.show()
```

21. Plot MAE

```
plt.figure(figsize=(8, 5))
plt.plot(history.history["mae"], label="Train MAE")
plt.plot(history.history["val_mae"], label="Validation MAE")
plt.xlabel("Epoch")
plt.ylabel("Mean Absolute Error")
plt.title("Model MAE")
plt.legend()
plt.show()
```

22. Evaluate Neural Network

```
mse_nn, mae_nn = model.evaluate(X_test, y_test)

print("Neural Network Results")
print("Mean Squared Error:", mse_nn)
print("Mean Absolute Error:", mae_nn)
print("RMSE:", np.sqrt(mse_nn))
```

23. Neural Network R2 Score

```
y_pred_nn = model.predict(X_test)
r2_nn = r2_score(y_test, y_pred_nn)

print("Neural Network R2 Score:", r2_nn)
```

24. Comparison

```
print("Linear Regression")
print("MSE:", mse_lr)
print("MAE:", mae_lr)
print("RMSE:", rmse_lr)
print("R2 Score:", r2_lr)

print("\nDeep Neural Network")
print("MSE:", mse_nn)
print("MAE:", mae_nn)
print("RMSE:", np.sqrt(mse_nn))
print("R2 Score:", r2_nn)
```

25. Predict New House Price

```
new_data = [[0.1, 10.0, 5.0, 0, 0.4, 6.0, 50, 6.0, 1, 400, 20, 300, 10]]

new_data_scaled = scaler.transform(new_data)

prediction = model.predict(new_data_scaled)

print("Predicted house price:", prediction[0][0])
```